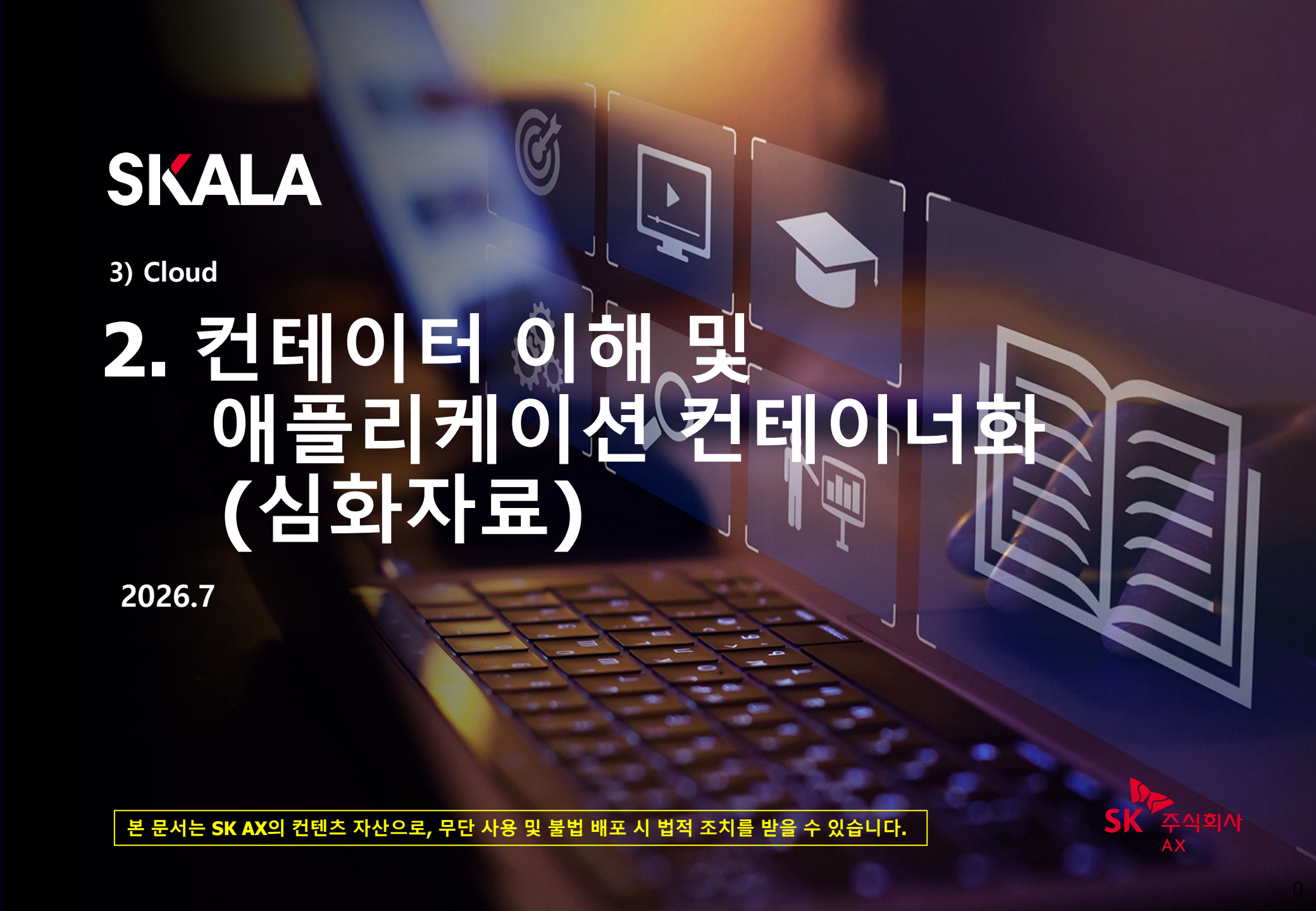




SKALA

3) Cloud

2. 컨테이너 이해 및 애플리케이션 컨테이너화 (심화자료)

2026.7

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

이런 분들을 위한 자료입니다



어느 정도 *Docker*를 다뤄본 경험이 있는 분들을 대상으로 합니다.

- 본 과정보다 빠르게 진도가 나가거나, 종합실습을 일찍 끝낸 분들을 위한 선택 학습 자료입니다.
- 필수 진도가 아니므로, 관심 있는 주제만 골라 자율적으로 학습하셔도 됩니다.
- 각 주제는 개념 설명과 실습이 있어서 직접 손으로 확인할 수 있도록 구성했습니다.

네 가지 심화 주제

순서와 상관없이, 관심 있는 주제부터 골라 학습하셔도 좋습니다.



SKALA 3) Cloud – 2. 컨테이너 이해 및 애플리케이션 컨테이너화

1. Dockerfile, 이미지 심화

- 빌드를 더 빠르고, 이미지를 더 작고 안전하게 만드는 기법들

Multi-stage Build, 한 걸음 더

 **기본 과정에서 배운 2단계(*build* → 운영) 구조를, 더 세분화할 수 있습니다.**

- 스테이지를 3개 이상으로 나눠, 의존성 설치 / 테스트 / 최종 빌드를 분리 가능
- 각 스테이지는 이름을 붙여(AS build, AS test) 다른 스테이지에서 선택적으로 참조
- `docker build --target` 옵션으로 특정 스테이지까지만 빌드해 디버깅 가능

3단계 Multi-stage Build 예시



```
FROM python:3.10 AS deps
```

```
WORKDIR /app
```

```
COPY requirements.txt .
```

```
RUN pip install --user -r requirements.txt
```

```
FROM deps AS test
```

```
COPY . .
```

```
RUN pytest
```

```
FROM python:3.10-slim AS final
```

```
COPY --from=deps /root/.local /root/.local
```

```
COPY . .
```

```
CMD ["python", "app.py"]
```



docker build --target test . 로 실행하면 테스트 단계까지만 검증 가능

BuildKit이란 무엇인가

 ***Docker의 차세대 빌드 엔진으로, 지금은 기본으로 활성화되어 있습니다.***

- 여러 레이어를 병렬로 빌드해 속도 향상
- 빌드 캐시를 더 정교하게 관리하고, 필요 없는 스테이지는 아예 Skip
- DOCKER_BUILDKIT=1 환경 변수로 활성화하거나(구버전), 최신 Docker는 기본값

캐시 마운트 — 라이브러리 캐시 재사용하기

BuildKit의 `--mount=type=cache`로, `pip/npm` 캐시를 레이어 밖에 별도로 보관할 수 있습니다.

```
# syntax=docker/dockerfile:1
FROM python:3.10-slim
WORKDIR /app
COPY requirements.txt .
RUN --mount=type=cache,target=/root/.cache/pip \
    pip install -r requirements.txt
```

💡 일반 레이어 캐시와 달리, 이 캐시는 이미지 레이어에 남지 않으면서도 빌드마다 재사용

.dockerignore, 기본 과정보다 한 걸음 더



기본

- node_modules, .git 제외
- *.log 제외
- 불필요한 파일 COPY 방지



심화

- .env, *.pem, *.key 같은 민감 파일도 반드시 제외
- 테스트 코드·문서(docs/, tests/)도 운영 이미지에서는 제외 고려
- 패턴은 .gitignore와 유사한 문법 사용 가능

Distroless 이미지



Distroless 이미지

“ 비유로 말하면

가구 하나 없이 침대만 있는 방입니다 — 필요한 것만 딱 있고, 불필요한 게 없습니다.

☰ 정확히 말하면

셸(bash 등)이나 패키지 매니저조차 없는 최소한의 이미지
공격자가 침투해도 활용할 도구 자체가 없어 보안에 유리

***gcr.io/distroless/python3* 처럼 구글이 공식 배포하는 이미지들이 있습니다.**

이미지 취약점, 왜 스캔해야 할까

 **베이스 이미지 안에는 우리가 모르는 오래된 라이브러리가 있을 수 있음**

- 이미지를 그대로 배포하면, 그 취약점도 함께 배포
- Trivy, Grype 같은 오픈소스 스캐너로 이미지를 스캔해 알려진 취약점(CVE)을 미리 확인 가능
- CI/CD 파이프라인에 스캔 단계를 넣으면, 취약한 이미지가 배포되기 전에 자동으로 필터링

[자율실습] Trivy로 이미지 스캔하기

🕒 무료 오픈소스 스캐너 Trivy로 우리가 만든 이미지를 검사

- ❶ `brew install trivy` (또는 공식 문서의 설치 방법)로 Trivy를 설치한다
- ❷ `trivy image my-was:1.0` 을 실행한다
- ❸ 출력되는 CVE 목록에서 심각도(CRITICAL, HIGH)를 확인한다
- ❹ 베이스 이미지를 더 최신 버전으로 바꾸면 취약점이 줄어드는지 비교해본다

✅ 베이스 이미지를 최신으로 유지하는 것만으로도 상당수의 취약점을 예방

멀티 아키텍처, 왜 필요한가

 **인텔(AMD64) 서버 외, Apple Silicon, ARM 기반 서버(AWS Graviton 등)**

- **Mac(ARM)에서 빌드한 이미지가 인텔 서버에서 안 돌아가는 경우도 있음**
- 하나의 이미지 태그 아래에, 여러 아키텍처용 이미지를 함께 묶어두는 것이 '멀티 아키텍처 이미지'

docker buildx



docker buildx

“ 비유로 말하면

한 벌의 옷을, 여러 사이즈로 동시에 만들어주는 재봉틀

☰ 정확히 말하면

Docker의 확장 빌드 도구

여러 CPU 아키텍처(AMD64, ARM64 등)를 위한 이미지를 한 번에 빌드하고 묶어서 업로드 가능

[자율 실습] buildx로 멀티 아키텍처 이미지 빌드

```
# 빌더 인스턴스 생성 (최초 1회)
```

```
$ docker buildx create --use
```

```
# AMD64, ARM64 두 아키텍처를 동시에 빌드 후 push
```

```
$ docker buildx build --platform linux/amd64,linux/arm64 \  
-t <registry>/my-was:1.0 --push .
```

💡 **--push** 없이 로컬에만 두고 싶다면 **--load**를 쓰되, 이 경우 단일 플랫폼만 가능

점검 항목	왜 중요한가
경량 베이스 이미지 사용	이미지 크기, 공격 표면 감소
Multi-stage build 적용	빌드 도구가 운영 이미지에 남지 않음
.dockerignore 정비	민감 파일 유출 방지, 빌드 속도 향상
취약점 스캔 습관화	알려진 CVE 를 배포 전에 차단
태그 버전 명시	롤백과 배포 이력 관리 용이

정리

✓ Multi-stage build는
3단계 이상으로도 세분화 가능

✓ Distroless 이미지는
공격 표면을 최소화

✓ buildx로 여러 CPU 아키텍처용
이미지를 한 번에 빌드 가능

✓ BuildKit과 캐시 마운트로
빌드 속도 향상

✓ Trivy 같은 도구로 이미지의
알려진 취약점을 스캔 가능

SKALA 3) Cloud – 2. 컨테이너 이해 및 애플리케이션 컨테이너화

2. 컨테이너 내부 동작 원리

- 실제 커널 명령어로 컨테이너의 원리를 직접 체험

Namespace 종류, 자세히 보기

기본 과정에서는 '칸막이'로 뭉뚱그려 배웠다면, 이번엔 종류별로 나눠봅니다.

Namespace 종류	격리 대상
PID	프로세스 번호 체계
Network	네트워크 인터페이스, IP , 포트
Mount	파일 시스템 마운트 지점
UTS	호스트 이름 (hostname)
IPC	프로세스 간 통신 (공유 메모리 등)
User	사용자 그룹 ID

[자율 실습] unshare로 나만의 Namespace 만들기

Docker 없이도, 리눅스 명령어만으로 격리된 공간을 만들 수 있습니다.

```
# 새로운 PID, Mount, UTS Namespace를 가진 셸 실행
```

```
$ sudo unshare --pid --mount --uts --fork bash
```

```
# 새 셸 안에서 호스트 이름을 바꿔본다
```

```
$ hostname my-namespace
```

```
$ hostname
```

```
# 다른 터미널(원래 셸)에서 hostname을 확인하면 그대로임을 확인
```

💡 Docker가 내부적으로 사용하는 것과 같은 원리 – Docker는 이 과정을 자동화해준 것

[자율 실습] nsenter로 컨테이너 내부 들여다보기

docker exec이 실제로는 무엇을 하는지, 더 낮은 레벨에서 확인

실행 중인 컨테이너의 PID 확인

\$ docker inspect --format '{{.State.Pid}}' my-was

→ 12345 (예시)

해당 프로세스의 namespace로 직접 진입

\$ sudo nsenter --target 12345 --pid --mount --net bash

💡 **docker exec은 nsenter와 비슷한 원리로 동작 — 실행 중 컨테이너의 namespace로 진입**

cgroups v2, 조금 더 깊이



실제 파일 시스템 레벨에서 확인

- 리눅스는 **/sys/fs/cgroup** 아래에 가상의 파일로 자원 제한을 표현
- cgroups v2에서는 이 계층 구조가 통합,
하나의 트리로 CPU, 메모리, I/O를 함께 관리

[자율 실습] 메모리 제한을 걸고 일부러 OOM 유발

🕒 제한을 넘어서면 실제로 무슨 일이 일어나는지 눈으로 점검

- ❶ `docker run --memory=20m --name oom-test python:3.10-slim python -c "a=[0]*10**8"` 을 실행
- ❷ `docker ps -a` 로 컨테이너 상태를 확인 (Exited 코드 137 확인)
- ❸ `docker inspect oom-test --format '{{.State.OOMKilled}}'` 로 OOM으로 종료됐는지 확인
- ❹ `--memory` 값을 넉넉하게 늘려 다시 실행해 정상 동작을 비교

✅ 종료 코드 **137**은 커널이 강제로 프로세스를 죽였다는 신호(**SIGKILL, 128+9**)



OverlayFS

“ 비유로 말하면

여러 장의 투명 필름을 겹쳐서 하나의 그림처럼 보이게 하는 것

☰ 정확히 말하면

이미지의 여러 레이어를 하나로 겹쳐 보여주는 리눅스 파일시스템
실제로는 여러 층이지만, 컨테이너 입장에서는 하나의 파일시스템

[자율 실습] OverlayFS 레이어 직접 확인하기



```
$ docker inspect my-was --format '{{json .GraphDriver}}' | python3 -m json.tool
```

출력 예시

```
{  
  "Data": {  
    "LowerDir": "/var/lib/docker/overlay2/.../diff:...",  
    "MergedDir": "/var/lib/docker/overlay2/.../merged",  
    "UpperDir": "/var/lib/docker/overlay2/.../diff"  
  }  
}
```



LowerDir(읽기 전용 레이어들), **UpperDir**(현재 컨테이너가 쓴 변경분),
MergedDir(합쳐진 최종 모습)로 구성

containerd와 runc의 관계



containerd

- 이미지 pull, 컨테이너 생명주기 관리
- Docker Daemon 내부적으로 사용
- Kubernetes의 기본 런타임



runc

- 실제로 Namespace·cgroups를 만들어 프로세스를 격리 실행
- OCI 표준을 구현한 저수준 실행기
- containerd가 이 runc를 호출해 컨테이너를 실행

OCI 표준, 다시 보기

Open Container Initiative — 컨테이너 생태계가 서로 호환되는 약속

- 이미지 포맷(**Image Spec**)과 실행 방식(**Runtime Spec**)
→ 두 가지를 표준화
- Docker로 만든 이미지를 Podman이나 Kubernetes에서도 그대로 실행 가능

정리

- ✓ **Namespace**는 **PID, Network, Mount** 등 6가지 이상으로 세분화
- ✓ **cgroups**는 **/sys/fs/cgroup** 파일 시스템으로 표현
- ✓ **OverlayFS**는 여러 레이어를 하나로 합쳐 보여주는 파일시스템
- ✓ **unshare, nsenter**로 **Docker** 없이도 격리 원리를 확인 가능
- ✓ 메모리 제한 초과 시 **OOM Killer**가 프로세스를 강제 종료(**exit 137**)
- ✓ **containerd**가 **runc**를 호출해 실제 컨테이너를 실행

SKALA 3) Cloud – 2. 컨테이너 이해 및 애플리케이션 컨테이너화

3. 운영 관점 — 보안, 리소스

- 실 환경에서의 보안과 자원 관리 이야기

컨테이너 보안, 기본 원칙

-  **컨테이너는 격리되어 있지만, 완벽히 안전한 것은 아닙니다.**
- 최소 권한 원칙 — 필요한 권한만 부여
 - 최소 이미지 원칙 — 불필요한 도구가 없을수록 공격 표면 감소
 - 최신 유지 원칙 — 베이스 이미지와 라이브러리를 주기적으로 업데이트

루트리스 컨테이너 (Rootless)



루트리스 컨테이너 (Rootless)

“ 비유로 말하면

손님에게 집 전체 열쇠 대신, 손님방 열쇠만 주는 것

☰ 정확히 말하면

컨테이너 안의 프로세스가 관리자(root) 권한 없이 실행되도록 하는 방식
컨테이너가 뚫려도 피해 범위를 최소화

[자율 실습] --user 옵션으로 비루트 사용자 실행하기

🕒 컨테이너를 root가 아닌 일반 사용자 권한으로 실행해봅니다.

- ❶ `docker run --user 1000:1000 -it python:3.10-slim whoami` 를 실행
- ❷ `id` 명령으로 UID/GID가 1000인지 확인
- ❸ Dockerfile에 USER 명령을 추가해 이미지 자체에서 기본 사용자를 지정하는 방법도 확인

✓ Dockerfile 마지막에 USER appuser 한 줄을 추가하면, 매번 --user를 안 써도 기본으로 적용됩니다.

읽기 전용 파일 시스템

 컨테이너 내부 파일이 실행 중에 변경되지 못하게 막을 수 있습니다.

- **--read-only** 옵션을 주면 루트 파일시스템이 읽기 전용
- 로그나 임시 파일처럼 쓰기가 꼭 필요한 경로는 **--tmpfs** 나 볼륨으로 별도 허용

[자율 실습] --read-only 옵션 실습

◎ 읽기 전용 컨테이너에서 쓰기를 시도하면 어떤 일이 생기는지 확인합니다.

- ❶ `docker run --read-only -it ubuntu bash` 로 실행
- ❷ `touch test.txt` 를 시도해 쓰기가 거부되는지 확인
- ❸ `docker run --read-only --tmpfs /tmp -it ubuntu bash` 로 다시 실행
/tmp 에는 쓰기가 가능한지 확인

Secrets 관리 — 환경 변수의 한계

 우리가 지금까지 써온 환경 변수(-e)는 사실 완전히 안전하지 않습니다.

- **docker inspect** 명령으로 환경 변수 값이 그대로 노출될 수 있음
- 비밀번호, API 키 같은 민감한 값은 별도의 Secrets 관리 도구로 다루는 것이 안전

환경 변수 vs Secrets



환경 변수 (-e)

- docker inspect로 그대로 노출됨
- 설정은 간단하지만 보안에 취약
- 간단한 실습, 개발 환경에 적합



Secrets (Compose/Swarm)

- 암호화되어 저장
- 필요한 컨테이너에만 전달
- 파일 형태로 컨테이너 내부에 마운트
- 운영 환경에 더 적합

리소스 제한, 실전처럼 다루기

 **기본 과정에서 배운 `--memory`, `--cpus`를 실제 운영 관점에서 다시 점검**

- 제한을 너무 낮게 걸면 서비스가 자주 죽고, 너무 높게(또는 안) 걸면 한 컨테이너가 서버 전체를 다운시킬 수 있음
- 실제 운영에서는 모니터링 데이터를 보고 적정 값을 찾아가는 것이 일반적

[자율 실습] docker stats로 실시간 리소스 확인하기

🕒 실행 중인 컨테이너들의 자원 사용량을 실시간으로 관찰합니다.

- ❶ docker stats 를 실행해 CPU·메모리 사용량을 실시간으로 확인
- ❷ 부하를 발생시키는 간단한 스크립트를 컨테이너 안에서 실행
- ❸ --cpus 값을 바꿔가며 CPU 사용률 상한이 실제로 적용되는지 관찰

로깅 드라이버, 어떤 게 있을까

드라이버	특징
json-file (기본값)	컨테이너별 JSON 파일로 로그 저장
local	json-file 보다 압축·성능이 개선된 기본 대안
syslog / journald	리눅스 표준 로깅 시스템과 연동
fluentd / gelf	ELK, Grafana Loki 등 외부 로그 시스템과 연동

운영 환경에서는 로그를 컨테이너 밖 중앙 시스템에 모아 분석하는 것이 일반적

정리

- ✓ 최소 권한, 최소 이미지, 최신 유지, 세 가지가 보안의 기본 원칙
- ✓ `--read-only`로 파일 시스템 변조 방지
- ✓ `docker stats`로 실시간 자원 사용량을 확인하며 제한값을 조정

- ✓ `--user`로 루트가 아닌 사용자로 컨테이너를 실행
- ✓ 민감한 값은 환경 변수 대신 `Secrets`로 관리하는 것이 더 안전
- ✓ 운영 환경에서는 로그를 중앙 시스템으로 모으는 것이 일반적

SKALA 3) Cloud – 2. 컨테이너 이해 및 애플리케이션 컨테이너화

4. Docker Compose 심화

- 기본 과정에서 배운 Compose를, 실무에서 쓰는 방식까지 확장

여러 Compose 파일을 합칠 수 있다

 ***docker-compose.yml** 하나에 모든 걸 다 적을 필요는 없습니다.*

- 기본 설정(**docker-compose.yml**) + 환경별 추가 설정(**override** 파일)
- 개발 환경에서는 소스코드를 bind mount하고,
운영 환경에서는 빌드된 이미지만 쓰는 식으로 분리 가능

docker-compose.override.yml



docker-compose.override.yml

“ 비유로 말하면

기본 레시피 위에, 오늘날만 추가하는 특별 양념

☰ 정확히 말하면

같은 폴더에 이 이름의 파일이 있으면, docker compose는 기본 파일과 자동으로 합쳐서 읽음 (주로 로컬 개발 전용 설정 활용)

[자율 실습] override 파일로 개발 환경 구성하기



```
# docker-compose.override.yml
```

```
services:
```

```
  was:
```

```
    volumes:
```

```
      - ./was:/app
```

```
    environment:
```

```
      - FLASK_DEBUG=1
```

```
# docker-compose.yml은 그대로 두고,
```

```
# docker compose up 만 실행해도 두 파일이 자동으로 합쳐집니다
```

Profiles — 선택적으로 서비스 실행하기

 모든 서비스를 항상 다 켜고 싶지 않을 때가 있습니다.

- 예: 평소엔 필요 없지만, 가끔 관리자 도구(Adminer 등)만 켜고 싶은 경우
- 서비스에 profiles를 지정해두면, 특정 profile을 지정했을 때만 실행되게 가능

[자율 실습] Profiles 활용하기



```
services:
```

```
  was:
```

```
    build: ./was
```

```
  db:
```

```
    image: postgres:17
```

```
  adminer:
```

```
    image: adminer
```

```
    profiles: ["debug"]
```

```
# 평소 실행: was, db만 켜짐
```

```
$ docker compose up -d
```

```
# adminer까지 켜고 싶을 때
```

```
$ docker compose --profile debug up -d
```

Healthcheck란 무엇인가

 컨테이너가 '떠 있다'고 해서 '정상 동작 중'이라는 보장은 없습니다.

- **healthcheck**는 주기적으로 컨테이너 내부에 간단한 명령을 실행해 진짜 살아있는지 확인
- `depends_on`과 함께 쓰면, '진짜로 준비된' 서비스가 뜬 뒤에야 다음 서비스가 시작되도록 만들 수 있습니다.

[자율 실습] Healthcheck 작성하기

```
services:
  db:
    image: postgres:17
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      timeout: 3s
      retries: 5

  was:
    build: ./was
    depends_on:
      db:
        condition: service_healthy
```

💡 **was**는 **db**가 단순히 '떠 있는' 상태가 아니라, 진짜 '접속 가능한' 상태일 때 시작

Restart Policy 종류

정책	동작
no (기본값)	재시작하지 않음
on-failure	오류로 종료됐을 때만 재시작
always	어떤 이유로든 항상 재시작
unless-stopped	사용자가 직접 멈추지 않는 한 항상 재시작

운영 환경에서는 보통 *unless-stopped* 또는 *always*를 많이 사용

.env 파일로 환경별 설정 분리하기

 *비밀번호나 포트 번호처럼 환경마다 달라지는 값을 코드 밖으로 분리*

- .env 파일에 **KEY=VALUE** 형식으로 값을 적어두면, **Compose**가 자동으로 읽어들이니다.
- docker-compose.yml 안에서는 `${변수명}` 형태로 참조

[자율 실습] .env 파일 적용하기

🕒 환경 변수를 코드에서 분리해 관리해봅니다.

- ❶ .env 파일에 POSTGRES_PASSWORD=1234, WAS_PORT=5000 을 적는다
- ❷ docker-compose.yml에서 ports: - "\${WAS_PORT}:5000" 처럼 참조하도록 바꾼다
- ❸ docker compose config 로 실제 값이 잘 치환되는지 미리 확인한다
- ❹ docker compose up -d 로 정상 동작을 확인한다

✅ .env 파일은 반드시 .gitignore에 추가해 저장소에 올라가지 않게 할 것

정리

✓ override 파일로 환경별 설정을
기본 파일과 분리

✓ healthcheck로 서비스가
'진짜 준비됐는지'까지 확인 가능

✓ .env 파일로 민감한 값과
환경별 설정을 코드에서 분리

✓ profiles로 특정 상황에만
필요한 서비스를 선택적 실행 가능

✓ restart policy로 장애 시
자동 복구 방식을 설정 가능

되돌아보기

- ✓ **Dockerfile** 이미지 심화 — 더 빠르고, 작고, 안전한 이미지를 만드는 법
- ✓ 컨테이너 내부 동작 원리 — 실제 커널 기술
- ✓ 운영 관점 — 실제 서비스에서 신경 써야 할 보안과 자원 관리
- ✓ **Docker Compose** 심화 — 실무에서 쓰는 방식으로 **Compose** 확장하기



수고하셨습니다.

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.